LAB-8: PREDICT DIABETES- PIMA INDIANS DATASET

8 health measurements → predict whether a patient has diabetes (0 = healthy, 1 = diabetic). 768 rows, classic benchmark dataset.

Column Name	Description	Unit / Values
Pregnancies	Number of times the patient was pregnant	Count
Glucose	Plasma glucose concentration after 2 hours (OGTT test)	mg/dL
BloodPressure	Diastolic blood pressure	mm Hg
SkinThickness	Triceps skinfold thickness	mm
Insulin	2-hour serum insulin level	mu U/ml
BMI	Body Mass Index	kg/m²
DiabetesPedigreeFunction	Genetic risk score for diabetes	0 – 2.5
Age	Age of the patient	Years
Outcome	Target column → 1 = Diabetic, 0 = Healthy	Binary (0/1)

Step 1 — Load & Explore

```
# ■■ Load and Explore ■■
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as s
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (classification_report, confusion_matrix,
ConfusionMatrixDisplay, roc_auc_score, roc_curve)

# Download the dataset if not already present
!test -f diabetes.csv || wget https://raw.githubusercontent.com/plotly/d

df = pd.read_csv('diabetes.csv') # https://www.kaggle.com/datasets/ucim
print(df.shape) # (768, 9)
print(df['Outcome'].value_counts()) # 500 healthy, 268 diabetic
print(df.describe().round(2)) # stats: mean/std/min/max
# Distribution of each feature vs outcome
df.hist(bins=25, figsize=(14,10), color='steelblue', edgecolor='white')
plt.suptitle('Feature Distributions', fontsize=14); plt.tight_layout();
```

```
--2026-06-17 10:02:07--  https://raw.githubusercontent.com/plotly/datasets/mas
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.11
HTTP request sent, awaiting response... 200 OK
Length: 23873 (23K) [text/plain]
Saving to: 'diabetes.csv'
```

Step 2 — Clean (Replace Impossible Zeros)

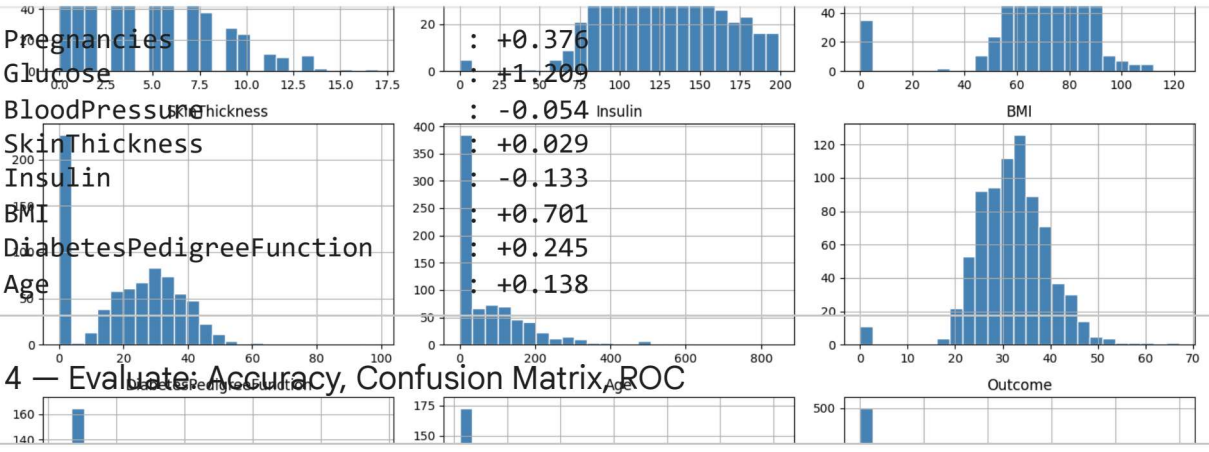
```
# ■■ Handle Missing Values ■■
# Glucose, BMI, etc. cannot be 0 — these are missing values disguised as zeros
fix_cols = ['Glucose','BloodPressure','SkinThickness','Insulin','BMI']
for col in fix_cols:
    n = (df[col] == 0).sum()
    df[col] = df[col].replace(0, df[col].median())
    print(f'{col:25s}: {n} zeros → replaced with median {df[col].median():.1f}'
```

```
Glucose                  : 5 zeros → replaced with median 117.0
BloodPressure            : 35 zeros → replaced with median 72.0
SkinThickness            : 227 zeros → replaced with median 23.0
Insulin                  : 374 zeros → replaced with median 31.2
BMI                      : 11 zeros → replaced with median 32.0
```

Step 3 — Split, Scale, Train

```
# ■■ Split, Scale, Train ■■
X = df.drop('Outcome', axis=1)
y = df['Outcome']
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.20, random_state=42, stratify=y # stratify keeps class ratio
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # fit on train, transform train
X_test = scaler.transform(X_test) # only transform test (no leakage!)
model = LogisticRegression(max_iter=1000, random_state=42)
model.fit(X_train, y_train)
# Print learned coefficient for each feature
for name, coef in zip(X.columns, model.coef_[0]):
    print(f'{name:30s}: {coef:+.3f}') # positive → increases diabetes risk
```

```
Pregnancies                    : +0.376
Glucose                        : +1.209
BloodPressure                  : -0.054
SkinThickness                  : +0.029
Insulin                        : -0.133
BMI                            : +0.701
DiabetesPedigreeFunction       : +0.245
Age                            : +0.138
```



Step 4 — Evaluate: Accuracy, Confusion Matrix, ROC

```
# ■■ Evaluate ■■
y_pred = model.predict(X_test)
```

```
y_prob = model.predict_proba(X_test)[:, 1]
print(classification_report(y_test, y_pred, target_names=['Healthy','Di
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
ConfusionMatrixDisplay(cm, display_labels=['Healthy','Diabetic']).plot(
plt.title('Diabetes Prediction – Confusion Matrix'); plt.show()
# ROC + AUC
fpr, tpr, _ = roc_curve(y_test, y_prob)
auc = roc_auc_score(y_test, y_prob)
plt.figure(figsize=(6.5, 4.5))
plt.plot(fpr, tpr, lw=2.5, color='steelblue', label=f'AUC = {auc:.3f}')
plt.plot([0,1],[0,1],'k--'); plt.xlabel('FPR'); plt.ylabel('TPR')
plt.title('Diabetes Model ROC Curve'); plt.legend(); plt.tight_layout()
# Expected: Accuracy ~78%, AUC ~0.84
```

```
                precision    recall  f1-score   support
```

Step 5 — Predict for a New Patient

```
       Healthy       0.75      0.82      0.78       100
```

```
# ■■ Predict New Patient + Coefficient Plot ■■
# Row: [Pregnancies, Glucose, BP, SkinThickness, Insulin, BMI, DPF, Age]
new_patient = np.array([[6, 148, 72, 35, 0, 33.6, 0.627, 50]])
new_scaled = scaler.transform(new_patient) # must use same scaler
pred = model.predict(new_scaled)[0]
prob = model.predict_proba(new_scaled)[0, 1]
print(f'Prediction : {'Diabetic' if pred==1 else 'Healthy'}')
print(f'Probability : {prob:.1%} chance of having diabetes')
# BONUS: Plot coefficients as a horizontal bar chart
import pandas as pd
coef_df = pd.DataFrame({'Feature': X.columns, 'Coefficient': model.coef_
coef_df = coef_df.sort_values('Coefficient')
coef_df.plot(kind='barh', x='Feature', y='Coefficient',
color=['tomato' if c > 0 else 'steelblue' for c in coef_df['Coefficient
legend=False, figsize=(7, 4))
plt.axvline(x=0, color='black', linewidth=0.8)
plt.title('Feature Coefficients — Positive = increases diabetes risk')
plt.tight_layout(); plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWa
  warnings.warn(
```